MoE 自定义 ONNX 节点导出与 RKNN 转换指南

发布版本：V1.2.0

日期：2026-10-29

修订记录

版本号	修改日期	修改说明
V1.2.0	2026-10-29	初始版本：MoE 自定义 ONNX 节点导出与 RKNN 转换指南。

本文介绍如何将 MoE 模型的路由专家部分（router、top-k、专家 FFN）导出为 `rknn::exUnifiedMoE`，再通过 RKNN3-Toolkit 的 `load_onnx` 或 `load_llm` 转换。

适用的 RKNN3-Toolkit 版本为 1.2.0 及以上。

1. 转换边界与验证顺序

```
HF/PyTorch MoE
  ├─ routed experts → ONNX rknn::exUnifiedMoE ─┐
  └─ shared expert  → 普通 ONNX 算子 ──────────┴→ load_onnx/load_llm → build →
export_rknn
```

`exUnifiedMoE` 只输出路由专家结果，形状和数据类型与 `hidden_states` 相同。shared expert 及其与路由专家结果的合并应保留在节点外，并遵循原模型的计算方式；否则结果会缺少 shared expert 的贡献。

建议先核对 `exUnifiedMoE` 的输入和属性，再截取单个 MoE 节点转换并对齐精度，最后转换完整模型并在目标设备验证。

2. `rknn::exUnifiedMoE` 的 ONNX 定义

建议使用 `domain="rknn"`，并在模型的 `opset_import` 中声明 `rknn`（示例使用版本 `1`），以便通过 `onnx.checker` 检查。Toolkit 按节点的 `op_type=exUnifiedMoE` 识别算子；示例中的 domain 版本不是算子的固定要求。

2.1 OP 输入输出

`exUnifiedMoE` 有一个输出。输入按下表含义**紧密排列**：未启用的可选项直接省略，不能用空字符串占位。记 `B=batch 大小`、`N=token 数`、`H=hidden_size`、`E=n_routed_experts`、`K=num_experts_per_token`、`I=专家中间维度`。目前暂时只支持 `B` 为 `1`。

顺序	输入	ONNX shape	数据类型	出现条件
1	`hidden_states`	`[1,N,H]`	float32	必需输入
2	`token_id_fix_experts`	`[B,N,K]`	int32	`token_id_fix_experts=1` 时传入固定专家 ID；仅支持仿真
3	`norm_before_router_scales`	`[H]`	float32	`norm_before_router_with_scales=1` 时传入
4	`router_weights`	`[H,E]`	float32	必需输入
5	`router_bias`	`[E]`	float32	`router_bias=1` 时传入
6	`norm_before_ffn_scales`	`[H]`	float32	`norm_before_ffn_with_scales=1` 时传入
7	`score_select_bias`	`[E]`	float32	`score_select_bias=1` 时传入；转换时须为常量
8	`per_expert_scale`	`[E]`	float32	`per_expert_scale=1` 时传入；转换时须为常量
之后	专家权重	见下表	float32	`expert_weight_packed='batch'` 时为一组，否则按专家编号循环堆叠表示 E 组专家输入

专家权重每组的顺序和形状：

`gate_up_packed`	组内输入顺序（启用 bias 才传）	非 batch 形状；`batch` 模式在每个形状前加 E 维
`0`	`gate_w`, `gate_b?`, `up_w`, `up_b?`, `down_w`, `down_b?`	`[H,I]`, `[I]`, `[H,I]`, `[I]`, `[I,H]`, `[H]`
`1`	`gate_up_w`, `gate_up_b?`, `down_w`, `down_b?`	`[H,2I]`; `[2I]`; `[I,H]`; `[H]`

`batch` 打包要求专家权重具有相同的 `I`，例如 `gate_up_w=[E,H,2I]`、`down_w=[E,I,H]`。所有矩阵权重均采用数学乘法 `x @ w` 的 `[in,out]` 布局。PyTorch `nn.Linear.weight` 通常为 `[out,in]`，应在**用户导出脚本**中转置；已经是 `[in,out]` 的权重不要再转置。bias 不转置。

`gate_up_cross_packed=0` 时，末维前半为 gate、后半为 up；设为 `1` 时，偶数位为 gate、奇数位为 up。交错模式要求 `gate_up_packed=1`，且 gate/up bias 同开或同关。

例如输入为

```
hidden_states
...
per_expert_scale
gate_w0
up_w0
down_w0
gate_w1
up_w1
down_w1
...
gate_we
up_we
down_we
```

2.2 OP 属性

以下为 **exUnifiedMoE 节点属性名**。使用 PyTorch legacy `symbolic` 时，在 `graph.op` 中给属性名加类型后缀，例如 `n_routed_experts_i=E`；导出后的属性名没有 `_i`。整数型开关请填入 `0/1`，不启用的属性直接省略，不要写成字符串 `"None"`。

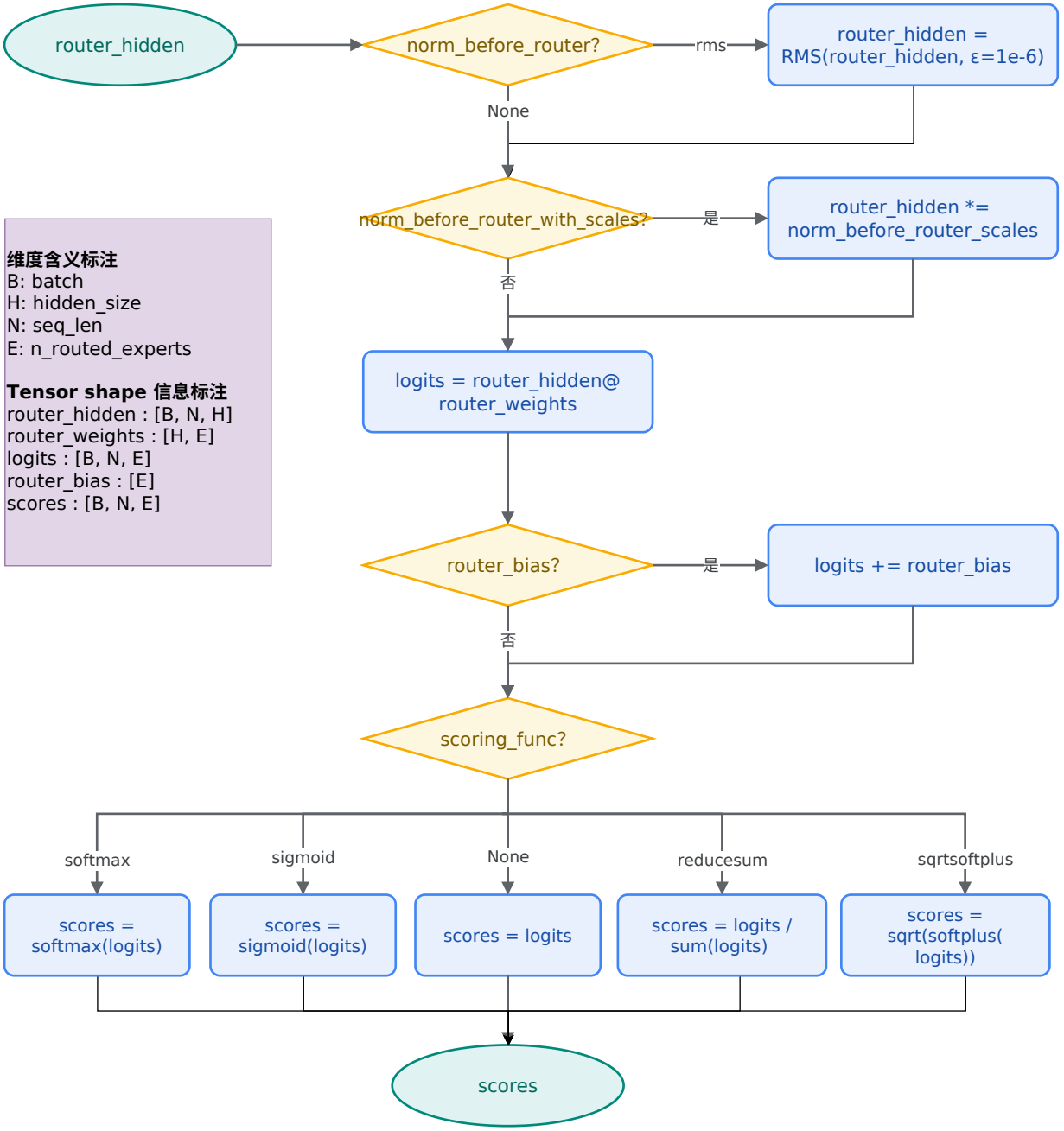
属性	值与作用
`n_routed_experts`	必填整数；路由专家数, 简写为 `E`，注意不含 shared expert。当前要求 `E` <= 2048
`num_experts_per_token`	必填整数；每个 token 选择的专家数，简写为 `K`, 数值范围为 `1 ≤ K ≤ E` 且 `K` <= 16
`topk_method`	`greedy`（默认）或 `group_limited_greedy`；后者还需设置 `n_group` 和 `topk_group`
`n_group`	使用 `group_limited_greedy` 时的分组数；须为正整数，且 `E % n_group == 0`
`topk_group`	使用 `group_limited_greedy` 时选中的组数；须满足 `1 ≤ topk_group ≤ n_group`、`K ≤ topk_group * E / n_group`
`scoring_func`	`softmax`、`sigmoid`、`reducesum`、`sqrtsoftplus`，或省略以直接使用 router logits；`reducesum` 表示按行求和归一化
`score_select_bias`	默认为 `0`；设为 `1` 时传入同名输入。bias 只影响 top-k 的专家 ID，专家权重仍取自未加 bias 的 score
`norm_selected_score`	`softmax`、`sigmoid`、`reducesum`、`sigmoid_reducesum`，或省略；作用于选中的 K 项
`routed_scaling_factor`	选中专家得分的缩放系数，默认 `1.0`
`norm_before_router`, `norm_before_ffn`	可设为 `rms`，分别作用于 router 输入和专家 FFN 输入；默认不归一化
`norm_before_router_with_scales`, `norm_before_ffn_with_scales`	默认为 `0`；设为 `1` 时须启用对应的 `rms` 并传入 `[H]` scale
`router_bias`, `gate_bias`, `up_bias`, `down_bias`	默认为 `0`；设为 `1` 时传入对应 bias
`expert_weight_packed`	`batch` 表示一组权重按 E 维堆叠；省略时传入 E 组独立权重

属性	值与作用
`gate_up_packed`, `gate_up_cross_packed`	默认为 `0`；分别表示合并 gate/up 权重、交错排列合并权重及 bias
`activation`	`silu` / `swish`、`gelu` / `gelu_erf`、`gelu_tanh`；默认 `silu`
`activation_alpha`	`silu` / `swish` 中 `x * sigmoid(alpha * x)` 的 `alpha`，默认 `1.0`
`gate_clip_mode`, `up_clip_mode`	分别控制 gate/up 投影结果的裁剪；`NONE`（默认）不裁剪，`MAX` 只限制上界，`MIN_MAX` 限制上下界；裁剪发生在 gate/up 相乘前
`gate_clip_min`, `up_clip_min`	gate/up 的裁剪下界；仅在对应模式为 `MIN_MAX` 时使用，默认 `0.0`
`gate_clip_max`, `up_clip_max`	gate/up 的裁剪上界；在对应模式为 `MAX` 或 `MIN_MAX` 时使用，默认 `0.0`
`up_offset`	裁剪后加到 up 的偏移，默认 `0.0`
`per_expert_scale`	默认为 `0`；设为 `1` 时传入同名输入
`routed_scores_before_ffn`	默认为 `0`，在 down 输出上乘路由得分；设为 `1` 时在专家 FFN 输入上乘路由得分
`token_id_fix_experts`	默认为 `0`；设为 `1` 时传入固定专家 ID，仅支持仿真

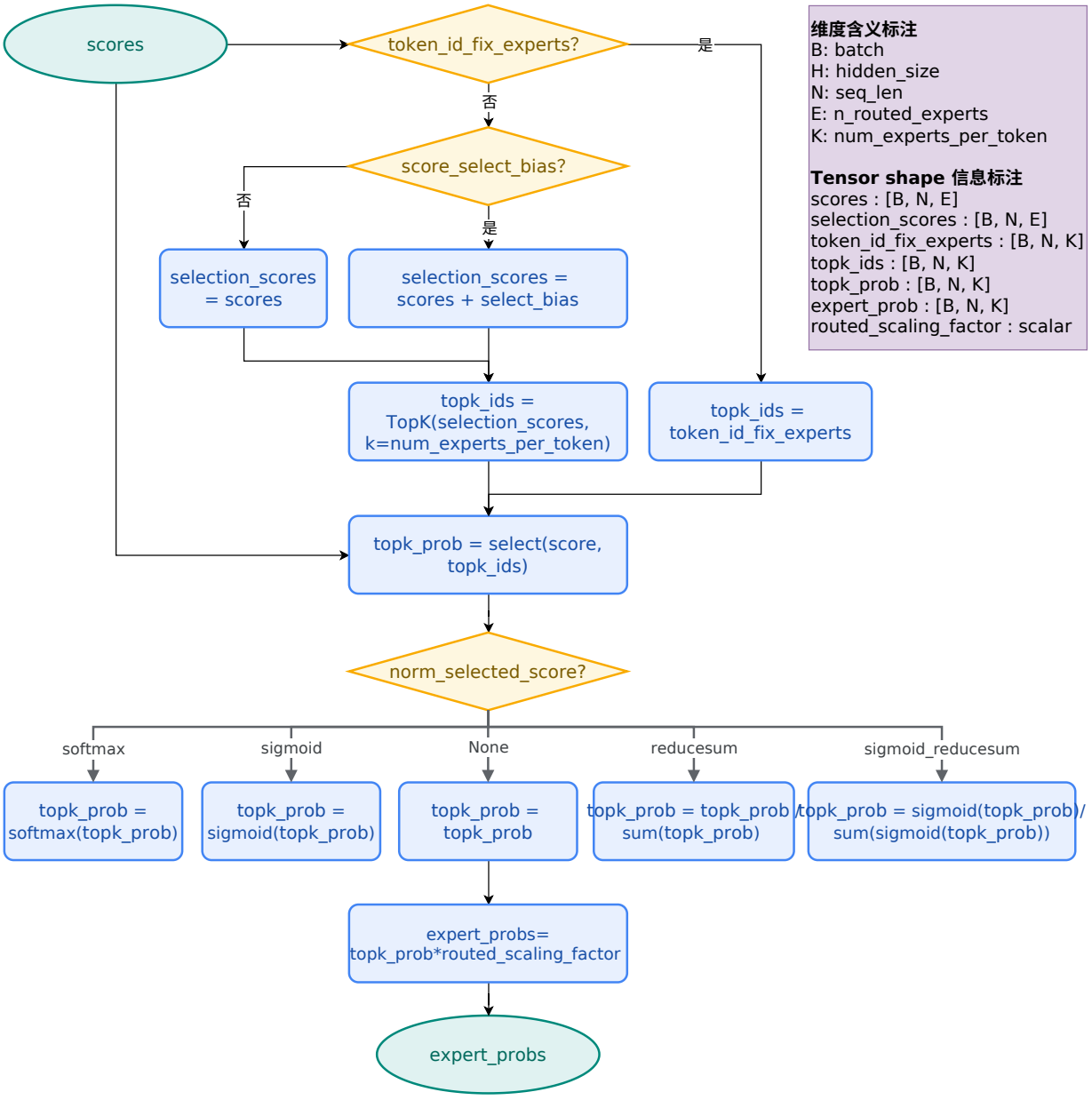
容量路由（`expert_capacity`、`expert_policy`）、固定专家 ID 路由(`token_id_fix_experts`)、`topk_method=group_limited_greedy`目前暂未完整支持，在生成 rknn 模型阶段会异常报错。

固定专家 ID 路由仅用于仿真：先在节点外按 token ID 查出专家 ID，维度信息为 `[B,N,K]`。每个 ID 须在 `[0,E)` 内。节点仍计算 router score，固定 ID 只替代 top-k 选取；选中得分仍按 `norm_selected_score` 和 `routed_scaling_factor` 处理。若原模型还输出 router logits，需在节点外保留对应路径。

2.3 scores 计算流程：



2.4 top-k 及专家权值计算流程



2.5 可直接生成的最小 ONNX

下面的脚本只依赖 `numpy` 和 `onnx`，用于检查基本输入顺序、权重布局和属性。它生成的是随机权重的**结构样例**，不能用于验证真实模型精度。保存为 `make_minimal_moe.py` 后运行 `python make_minimal_moe.py`。

```
import numpy as np
import onnx
from onnx import TensorProto, helper, numpy_helper

H, I, E, K, N = 8, 16, 4, 2, 3
rng = np.random.default_rng(0)

def weight(name, shape):
    array = rng.standard_normal(shape).astype(np.float32) * 0.01
    return numpy_helper.from_array(array, name=name)

# batch 打包: hidden, router, gate_up, down; 没有 bias/scale 时不留空输入。
initializers = [
    weight("router_w", (H, E)),
    weight("gate_up_w", (E, H, 2 * I)),
    weight("down_w", (E, I, H)),
]
node = helper.make_node(
    "exUnifiedMoE", ["hidden", "router_w", "gate_up_w", "down_w"],
    ["routed_out"], name="moe_0", domain="rknn",
    n_routed_experts=E, num_experts_per_token=K,
    topk_method="greedy", scoring_func="softmax",
    norm_selected_score="reducesum", expert_weight_packed="batch",
    gate_up_packed=1, activation="silu",
)
graph = helper.make_graph(
    [node], "minimal_moe",
    [helper.make_tensor_value_info("hidden", TensorProto.FLOAT, [1, N, H])],
    [helper.make_tensor_value_info("routed_out", TensorProto.FLOAT, [1, N, H])],
    initializer=initializers,
)
model = helper.make_model(
    graph, opset_imports=[helper.make_operatorsetid("", 19),
                          helper.make_operatorsetid("rknn", 1)],
)
onnx.checker.check_model(model)
onnx.save_model(model, "minimal_moe.onnx")
```

2.6 从 HF/PyTorch 导出真实模型

在模型导出脚本中，可用 `torch.autograd.Function` 封装路由专家部分：`forward` 保留与原模型等价的计算，用于导出前对拍；`symbolic` 生成 `rknn::exUnifiedMoE`。调用 `Function.apply` 前，按第 2 节将权重排成 `[in,out]`；不要在 `symbolic` 中对 ONNX `Value` 调用 PyTorch Tensor 的 `.t()`。例如：

```python
@staticmethod
def symbolic(g, hidden, router_w, gate_up_w, down_w):
    out = g.op(
        "rknn::exUnifiedMoE", hidden, router_w, gate_up_w, down_w,
        n_routed_experts_i=E, num_experts_per_token_i=K,
        scoring_func_s="softmax", norm_selected_score_s="reducesum",
        expert_weight_packed_s="batch", gate_up_packed_i=1,
        activation_s="silu", outputs=1,
    )
    out.setType(hidden.type())
    return out
```

`E`、`K` 应从模型配置取得；`forward` 和模型替换逻辑需按具体模型实现。若原权重为 `gate_up_proj.weight=[E,2I,H]`、`down_proj.weight=[E,H,I]`，分别用 `transpose(1, 2).contiguous()` 转成 `[E,H,2I]`、`[E,I,H]`；router `Linear.weight=[E,H]` 用 `transpose(0, 1).contiguous()` 转成 `[H,E]`。若权重已是 `[in,out]` 布局，直接传入。逐项核对原模型的路由得分、归一化、bias 和 shared expert，不要仅凭模型名称选属性。

使用支持 `symbolic` 的 PyTorch legacy exporter，例如：

```python
torch.onnx.export(
    model, inputs, "model.onnx", opset_version=19,
    export_params=True, external_data=True,
    custom_opsets={"rknn": 1}, dynamo=False,
)
```

不同 PyTorch 版本的导出参数可能有差异；请检查**实际生成的 ONNX** 是否包含 `rknn::exUnifiedMoE`。若模型使用 external data，转换时需保留 ONNX 引用的全部权重文件。

若原模型的 gate/up 权重交错排列，应设置 `gate_up_cross_packed=1`；激活、裁剪、偏移和 bias 也须按原模型配置导出。路由专家以外的 mask 逻辑应留在节点外。

3. 不依赖 Toolkit 源码的检查与定位

3.1 转换一个 MoE 节点

`load_onnx(inputs=..., outputs=...)` 可截取子图。先找到所需 MoE 节点的 `input[0]` 和 `output[0]`；如需验证包含 shared expert 的完整模块，应选合并之后的张量作为输出。`input_size_list` 使用该输入的实际 `[1,N,H]`：

```
import onnx
from rknn.api import RKNN

path = "model.onnx"
model = onnx.load(path, load_external_data=False)
moe = next(n for n in model.graph.node if n.op_type == "exUnifiedMoE")
print("cut:", moe.name, moe.input[0], "->", moe.output[0])

N, H = 3, 8  # 改成当前 MoE 层的真实 token 数和 hidden_size
rknn = RKNN(verbose=True)
try:
    assert rknn.config(target_platform="rk1820", core_num=8,
                       enable_moe_tc=False) == 0
    assert rknn.load_onnx(model=path, inputs=[moe.input[0]],
                          input_size_list=[[1, N, H]],
                          outputs=[moe.output[0]]) == 0
    assert rknn.build(do_quantization=False) == 0
    assert rknn.export_rknn("single_moe.rknn") == 0
finally:
    rknn.release()
```

`rk1820` / `core_num` 应换成实际目标配置。若只验证路由节点，参考值应是原模型的路由专家输出。用相同输入和精度设置比较形状、最大绝对误差和余弦相似度；单层通过后，再验证整模型和目标设备。示例先关闭量化，量化模型需按实际配置构建并单独对拍。

整模型若符合 LLM 导入要求，可使用 `RKNN.config(..., llm_config=...)`、`load_llm("model.onnx")`、`build(...)`、`export_rknn(...)`。每步检查返回码并保存完整日志。

3.2 固定 shape 的有效 token 数

该功能暂未实装, 目前不支持!

`enable_moe_tc` 默认为 `False`。需要为固定形状的 prefill 指定有效 token 数时，可在确认目标环境支持后设为 `True`。启用成功后，导出模型会增加一个 `[1] int32` 输入（通常名为 `MoE_Tc`）；推理时按实际输入列表提供有效 token 数，范围为 `[0,N]`。

启用该功能要求模型中 MoE 节点的序列长度相同，且为已知正整数。`MoE_Tc` 无需加入原始 ONNX 的 `exUnifiedMoE` 输入表；初次对拍可保持默认设置。

3.3 常见现象与处理

现象/日志	优先检查
ONNX 中没有 `exUnifiedMoE` 节点	检查导出器是否调用 `symbolic`，以及是否替换了目标 MoE 层

现象/日志	优先检查
缺少必要属性，或提示 `hidden_states must be [1,N,H]`	检查 `E`、`K` 和输入的 batch/静态形状
输入数量不符	按第 2 节核对可选输入开关和专家组数，不要传空占位
专家权重形状不符	检查 `Linear` 权重转置、`batch` 首维 E、gate/up 的 `2I` 和 down 的 `[I,H]`
属性组合不支持，构建失败	对照第 2.1 节检查路由方式和属性取值，尤其是容量路由和固定专家 ID 路由
`per_expert_scale` 不是常量	将该 `[E]` scale 作为 ONNX initializer/常量导出
`score_select_bias` 不是 FP32 常量	将该 `[E]` bias 作为 FP32 initializer/常量导出
`MoE_Tc` 未能启用	检查各 MoE 节点的序列长度是否一致；初次对拍可关闭 `enable_moe_tc`
构建成功但结果偏差大	先比较单层未量化结果，再核对 score 归一化、gate/up 布局、激活与裁剪、shared expert 和外部 mask
只在目标设备失败	核对 Toolkit 和设备运行时版本，保留输入样本、模型及日志以便定位

反馈问题时，请提供 Toolkit 和设备运行时版本、目标芯片、可复现的 ONNX 及其 external data、节点属性和输入形状、转换日志，以及相同输入下原模型与 RKNN 的输出。若权重不能分享，可用同结构的随机权重模型复现导出问题；精度问题仍需用真实模型对拍。